

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра компьютерных и информационных наук

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 8

дисциплина: Архитектура компьютеров

Студент: Симакова Алина

Группа: НКАбд-05-25

МОСКВА

2025 г.

Оглавление

1 Цель работы.....	3
2 Задание.....	4
3 Теоретическое введение.....	5
4 Выполнение лабораторной работы.....	6
4.1 Реализация циклов в NASM.....	6
4.2 Обработка аргументов командной строки.....	9
5 Задания для самостоятельной работы.....	13
6 Выводы.....	16
Список литературы.....	17

1 Цель работы

Целью данной лабораторной работы является приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки.

2 Задание

На основе методических указаний осуществить реализацию циклов в NASM, изучить обработку аргументов командной строки и выполнить самостоятельное написание программ по материалам лабораторной работы.

3 Теоретическое введение

Стек — это структура данных, организованная по принципу LIFO («Last In — First Out» или «последним пришёл — первым ушёл»). Стек является частью архитектуры процессора и реализован на аппаратном уровне. Для работы со стеком в процессоре есть специальные регистры (ss, bp, sp) и команды.

Основной функцией стека является функция сохранения адресов возврата и передачи аргументов при вызове процедур. Кроме того, в нём выделяется память для локальных переменных и могут временно храниться значения регистров.

Стек имеет вершину, адрес последнего добавленного элемента, который хранится в регистре esp (указатель стека). Противоположный конец стека называется дном. Значение, помещённое в стек последним, извлекается первым. При помещении значения в стек указатель стека уменьшается, а при извлечении — увеличивается.

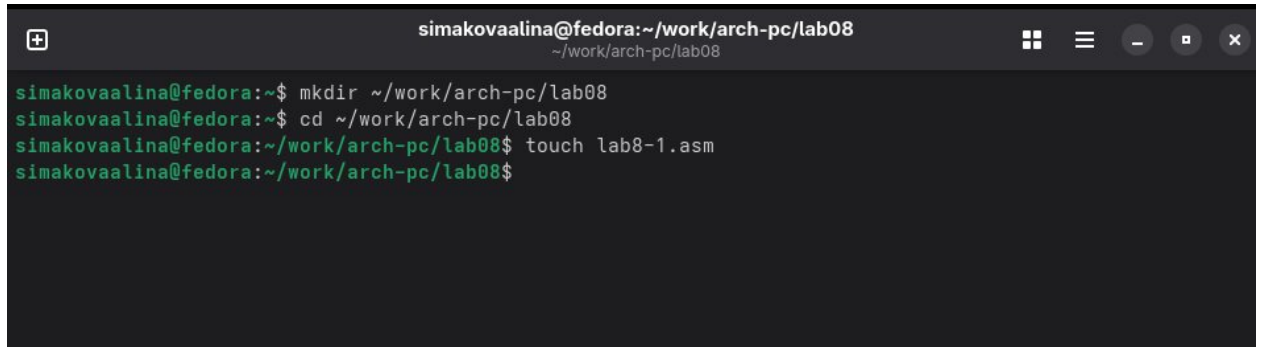
Для стека существует две основные операции:

- добавление элемента в вершину стека (push);
- извлечение элемента из вершины стека (pop).

4 Выполнение лабораторной работы

4.1 Реализация циклов в NASM

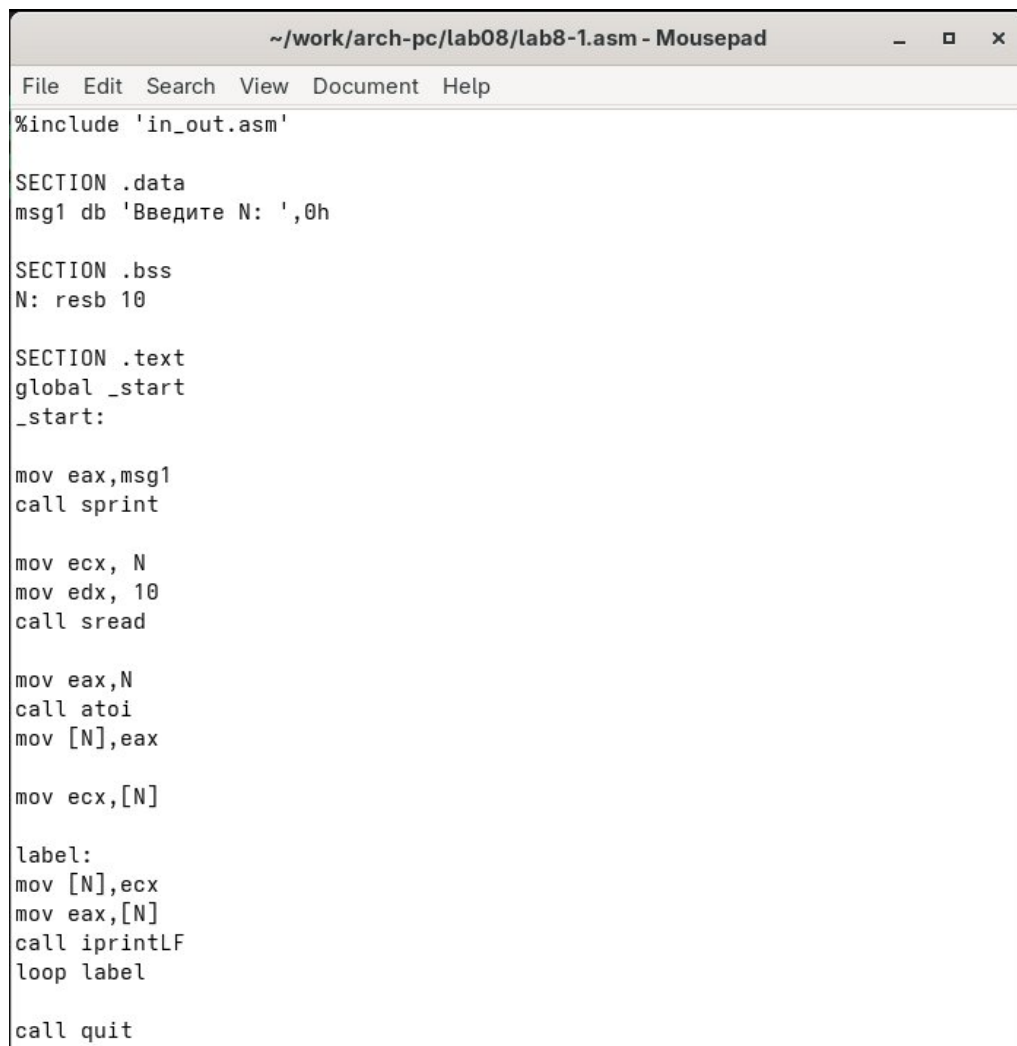
В домашней директории создаю каталог для программ лабораторной работы №8 (рис. 4.1).



```
simakovaalina@fedora:~/work/arch-pc/lab08
simakovaalina@fedora:~$ mkdir ~/work/arch-pc/lab08
simakovaalina@fedora:~$ cd ~/work/arch-pc/lab08
simakovaalina@fedora:~/work/arch-pc/lab08$ touch lab8-1.asm
simakovaalina@fedora:~/work/arch-pc/lab08$
```

Рис. 4.1: Создание каталога

Далее копирую в созданный файл программу из листинга (рис. 4.2).



```
~/work/arch-pc/lab08/lab8-1.asm - Mousepad
File Edit Search View Document Help
%include 'in_out.asm'

SECTION .data
msg1 db 'Введите N: ',0h

SECTION .bss
N: resb 10

SECTION .text
global _start
_start:

mov eax,msg1
call sprint

mov ecx, N
mov edx, 10
call sread

mov eax,N
call atoi
mov [N],eax

mov ecx,[N]

label:
mov [N],ecx
mov eax,[N]
call iprintLF
loop label

call quit
```

Рис. 4.2: Копирование программы из листинга

Запускаю программу, она показывает работу циклов в NASM (рис. 4.3).

```
simakovaalina@fedora:~/work/arch-pc/lab08$ mousepad lab8-1.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
simakovaalina@fedora:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 10
10
9
8
7
6
5
4
3
2
1
simakovaalina@fedora:~/work/arch-pc/lab08$
```

Рис. 4.3: Запуск программы

Заменяю программу таким образом, что в теле цикла, я изменяю значение регистра `eax` (рис. 4.4).

```
~/work/arch-pc/lab08/lab8-1.asm - Mousepad
File Edit Search View Document Help
%include 'in_out.asm'

SECTION .data
msg1 db 'Введите N: ', 0h

SECTION .bss
N: resb 10

SECTION .text
GLOBAL _start
_start:

mov eax, msg1
call sprint

mov ecx, N
mov edx, 10
call sread

mov eax, N
call atoi
mov [N], eax

mov ecx, [N]

label:
sub ecx, 1
mov [N], ecx
mov eax, [N]
call iprintLF
loop label

call quit
```

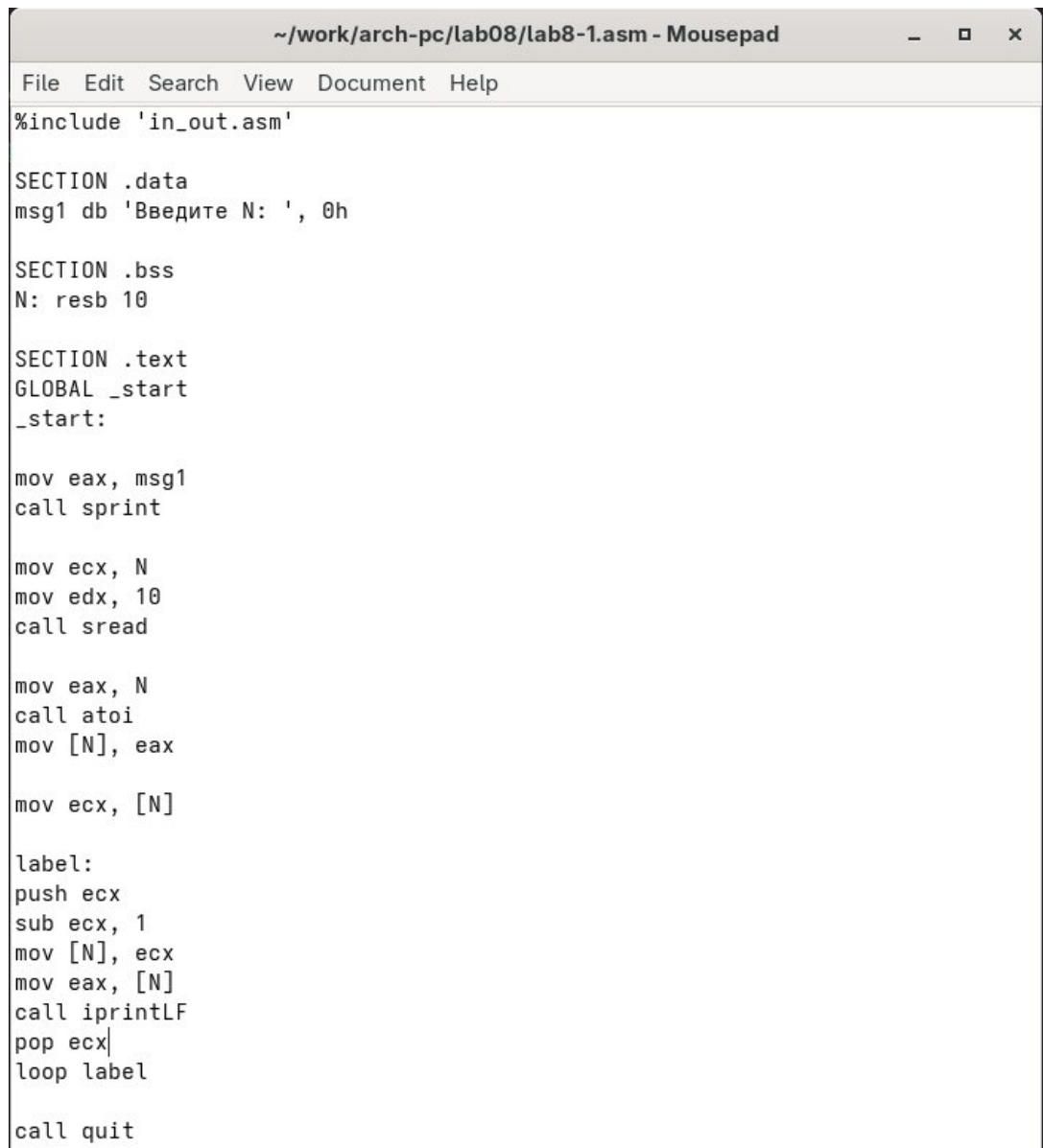
Рис. 4.4: Изменение программы

Из-за того, что теперь регистр `ecx` на каждой итерации уменьшается на 2 значения, количество итераций уменьшается вдвое (рис. 4.5).

```
simakovaalina@fedora:~/work/arch-pc/lab08$ mousepad lab8-1.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
simakovaalina@fedora:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 10
9
7
5
3
1
simakovaalina@fedora:~/work/arch-pc/lab08$
```

Рис. 4.5: Запуск измененной программы

Добавляю команды `push` и `pop` в программу (рис. 4.6).



```
~/work/arch-pc/lab08/lab8-1.asm - Mousepad
File Edit Search View Document Help
%include 'in_out.asm'

SECTION .data
msg1 db 'Введите N: ', 0h

SECTION .bss
N: resb 10

SECTION .text
GLOBAL _start
_start:

mov eax, msg1
call sprint

mov ecx, N
mov edx, 10
call sread

mov eax, N
call atoi
mov [N], eax

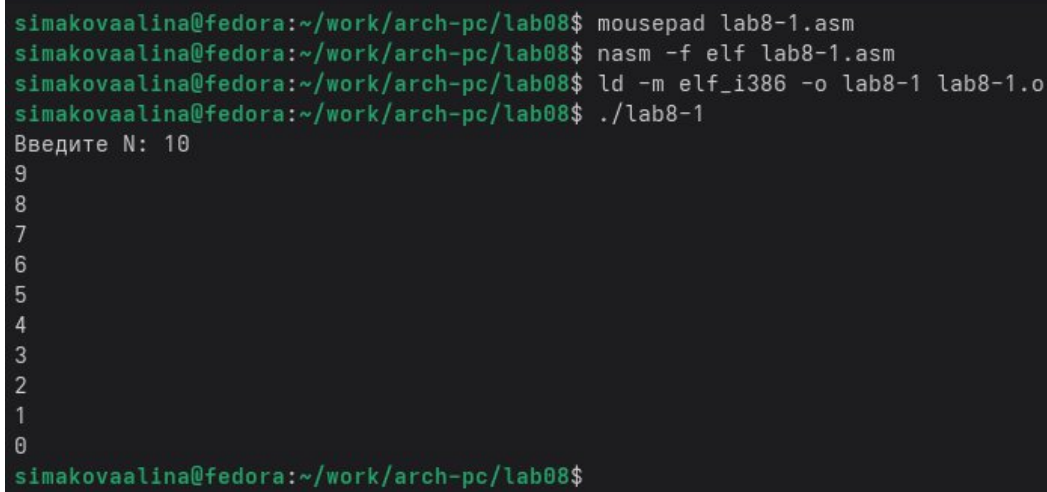
mov ecx, [N]

label:
push ecx
sub ecx, 1
mov [N], ecx
mov eax, [N]
call iprintLF
pop ecx
loop label

call quit
```

Рис. 4.6: Добавление `push` и `pop` в цикл программы

Теперь количество итераций совпадает введенному N, но происходит смещение выводимых чисел на -1 (рис. 4.7).

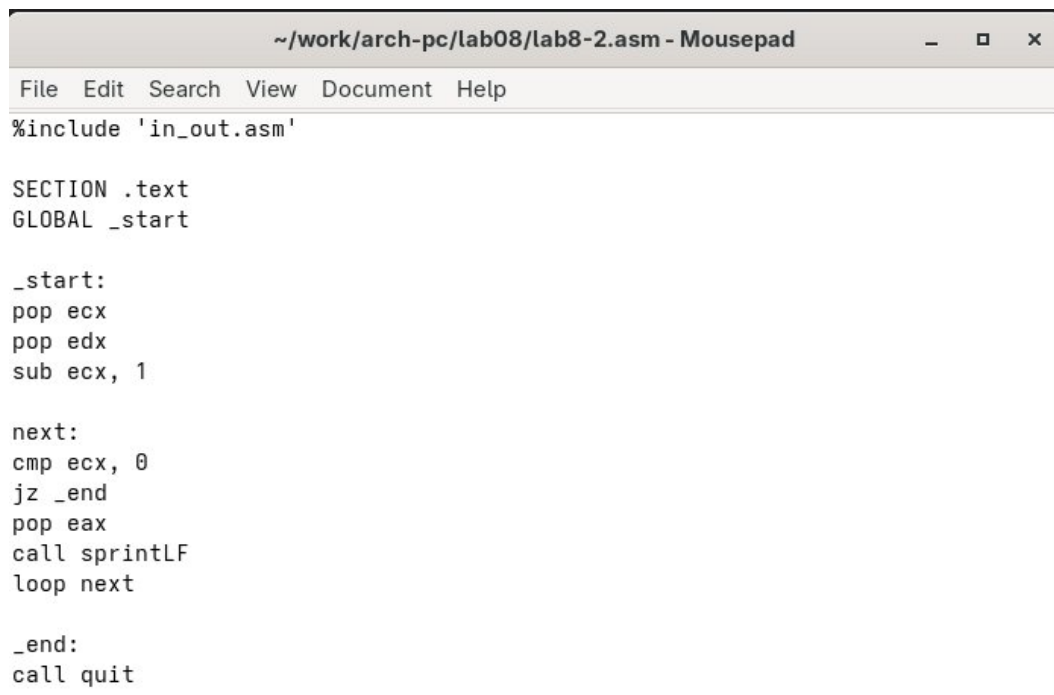


```
simakovaalina@fedora:~/work/arch-pc/lab08$ mousepad lab8-1.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
simakovaalina@fedora:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 10
9
8
7
6
5
4
3
2
1
0
simakovaalina@fedora:~/work/arch-pc/lab08$
```

Рис. 4.7: Запуск измененной программы

4.2 Обработка аргументов командной строки

Создаю новый файл для программы и копирую в него код из следующего листинга (рис. 4.8).



```
~/work/arch-pc/lab08/lab8-2.asm - Mousepad
File Edit Search View Document Help
%include 'in_out.asm'

SECTION .text
GLOBAL _start

_start:
    pop ecx
    pop edx
    sub ecx, 1

next:
    cmp ecx, 0
    jz _end
    pop eax
    call sprintf
    loop next

_end:
    call quit
```

Рис. 4.8: Копирование программы из листинга

Затем компилирую программу и запускаю, указав аргументы. Получилось так, что программой было обработано то же количество аргументов, что и было выведено (рис. 4.9).

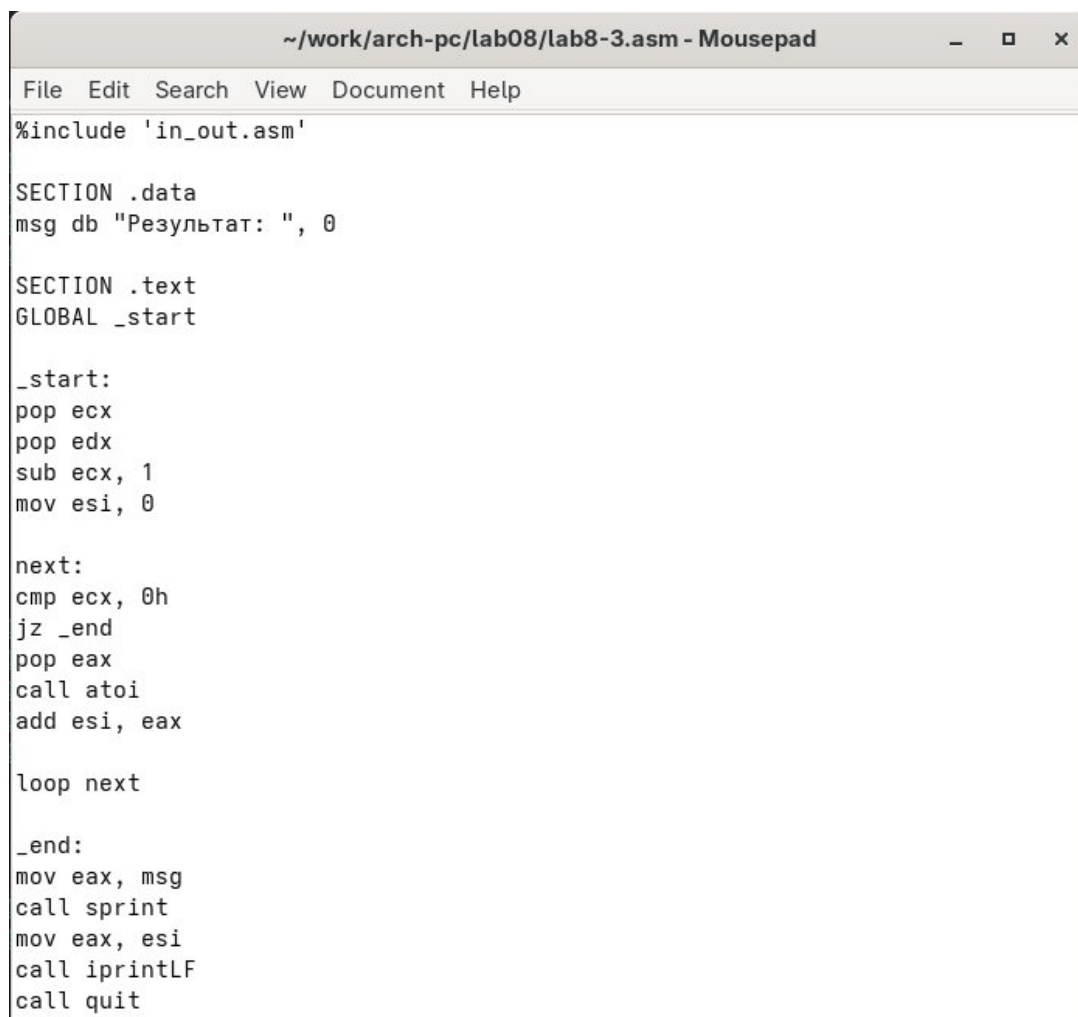
```

simakovaalina@fedora:~/work/arch-pc/lab08$ touch lab8-2.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ mousepad lab8-2.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-2.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-2 lab8-2.o
simakovaalina@fedora:~/work/arch-pc/lab08$ ./lab8-2 arg1 arg 2 'arg 3'
arg1
arg
2
arg 3
simakovaalina@fedora:~/work/arch-pc/lab08$

```

Рис. 4.9: Запуск второй программы

Создаю новый файл для программы и копирую в него код из третьего листинга (рис. 4.10).



```

~/work/arch-pc/lab08/lab8-3.asm - Mousepad
File Edit Search View Document Help
%include 'in_out.asm'

SECTION .data
msg db "Результат: ", 0

SECTION .text
GLOBAL _start

_start:
pop ecx
pop edx
sub ecx, 1
mov esi, 0

next:
cmp ecx, 0h
jz _end
pop eax
call atoi
add esi, eax

loop next

_end:
mov eax, msg
call sprint
mov eax, esi
call iprintLF
call quit

```

Рис. 4.10: Копирование программы из третьего листинга

Компилирую программу и запускаю, указав в качестве аргументов некоторые числа, программа их складывает (рис. 4.11).

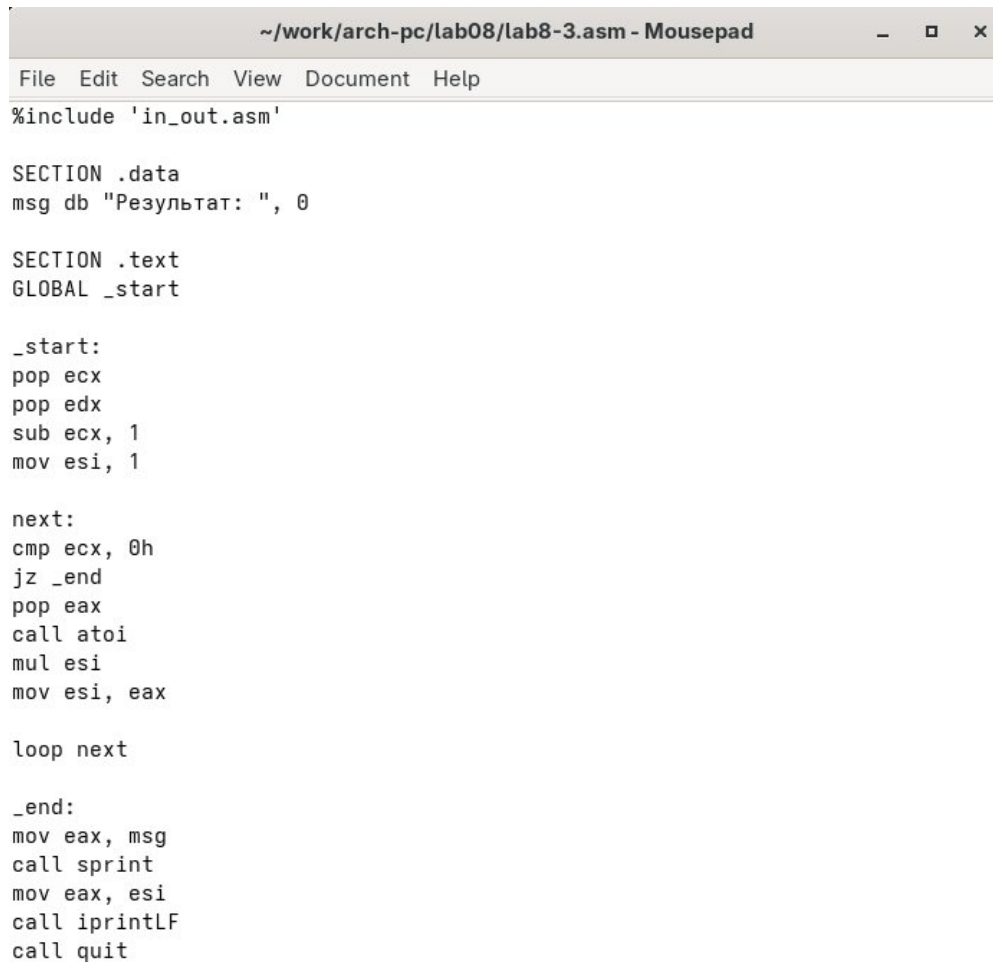
```

simakovaalina@fedora:~/work/arch-pc/lab08$ touch lab8-3.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ mousepad lab8-3.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
simakovaalina@fedora:~/work/arch-pc/lab08$ ./lab8-3 12 13 7 10 5
Результат: 47
simakovaalina@fedora:~/work/arch-pc/lab08$

```

Рис. 4.11: Запуск третьей программы

Изменяю поведение программы так, чтобы указанные аргументы она умножала, а не складывала (рис. 4.12).



```

~/work/arch-pc/lab08/lab8-3.asm - Mousepad
File Edit Search View Document Help
%include 'in_out.asm'

SECTION .data
msg db "Результат: ", 0

SECTION .text
GLOBAL _start

_start:
pop ecx
pop edx
sub ecx, 1
mov esi, 1

next:
cmp ecx, 0h
jz _end
pop eax
call atoi
mul esi
mov esi, eax

loop next

_end:
mov eax, msg
call sprint
mov eax, esi
call iprintLF
call quit

```

Рис. 4.12: Изменение третьей программы

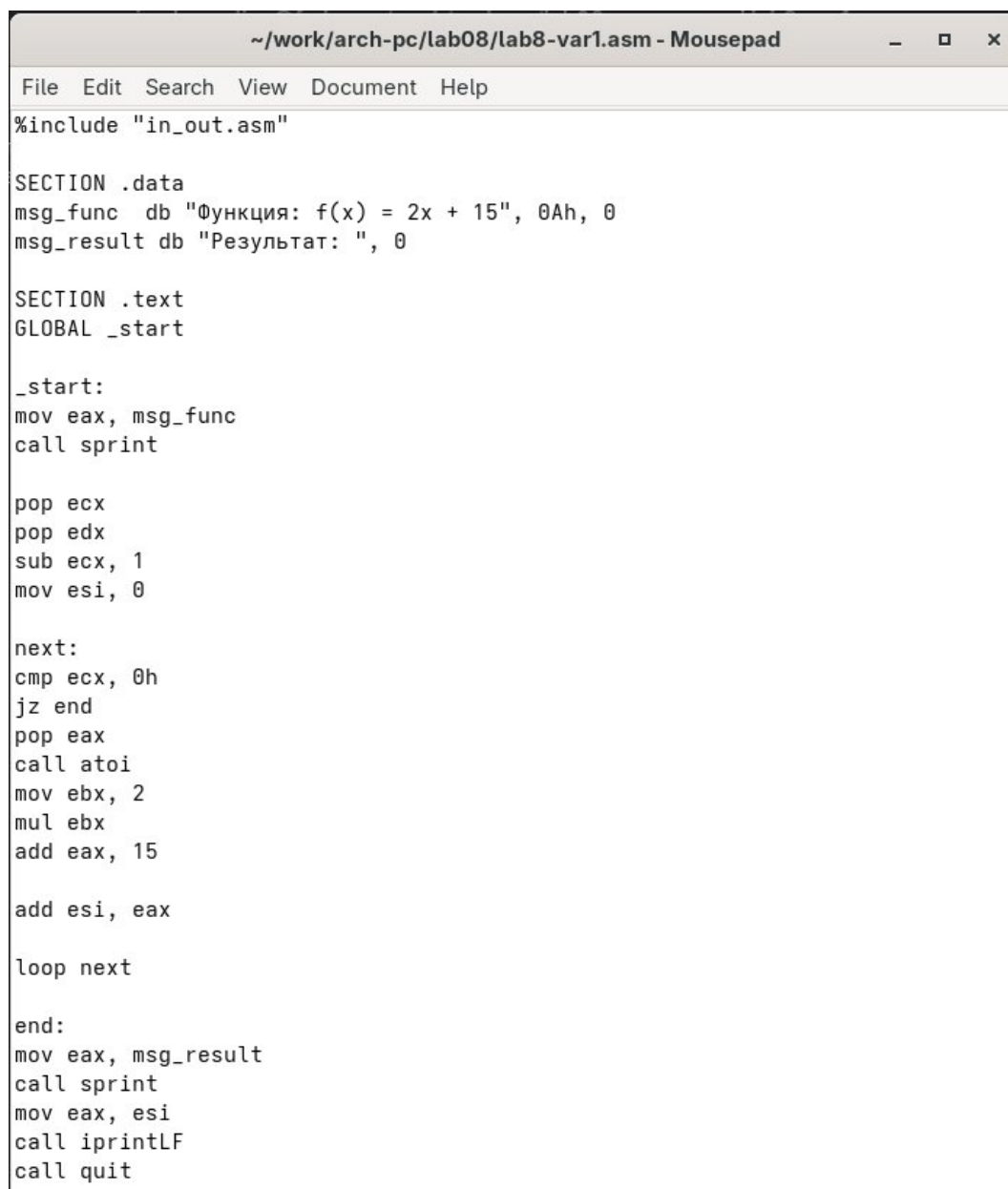
Я убедилась, что программа действительно теперь умножает данные на вход числа (рис. 4.13).

```
simakovaalina@fedora:~/work/arch-pc/lab08$ mousepad lab8-3.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
simakovaalina@fedora:~/work/arch-pc/lab08$ ./lab8-3 100 1 7
Результат: 700
simakovaalina@fedora:~/work/arch-pc/lab08$
```

Рис. 4.13: Запуск измененной третьей программы

5 Задания для самостоятельной работы

Пишу программу, которая будет находить сумму значений для функции $f(x) = 2x + 15$, которая соответствует моему варианту №1 (рис. 5.1).



```
~/work/arch-pc/lab08/lab8-var1.asm - Mousepad
File Edit Search View Document Help
%include "in_out.asm"

SECTION .data
msg_func db "Функция: f(x) = 2x + 15", 0Ah, 0
msg_result db "Результат: ", 0

SECTION .text
GLOBAL _start

_start:
mov eax, msg_func
call sprint

pop ecx
pop edx
sub ecx, 1
mov esi, 0

next:
cmp ecx, 0h
jz end
pop eax
call atoi
mov ebx, 2
mul ebx
add eax, 15

add esi, eax

loop next

end:
mov eax, msg_result
call sprint
mov eax, esi
call iprintLF
call quit
```

Рис. 5.1: Написание программы для самостоятельной работы

Прикладываю код первой программы:

```
%include "in_out.asm"
```

```
SECTION .data
```

```
msg_func db "Функция: f(x) = 2x + 15", 0
```

```
msg_result db "Результат: ", 0
```

SECTION .text

GLOBAL _start

_start:

mov eax, msg_func

call sprint

pop ecx

pop edx

sub ecx, 1

mov esi, 0

next:

cmp ecx, 0h

jz end

pop eax

call atoi

mov ebx, 2

mul ebx

add eax, 15

add esi, eax

loop next

end:

mov eax, msg_result

call sprint

mov eax, esi

call iprintLF

call quit

Затем проверяю работу программы, указав в качестве аргумента несколько чисел (рис. 5.2)

```
simakovaalina@fedora:~/work/arch-pc/lab08$ mousepad lab8-var1.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ nasm -f elf lab8-var1.asm
simakovaalina@fedora:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-var1 lab8-var1.o
simakovaalina@fedora:~/work/arch-pc/lab08$ ./lab8-var1 1 2 3 4
Функция:  $f(x) = 2x + 15$ 
Результат: 80
simakovaalina@fedora:~/work/arch-pc/lab08$
```

Рис. 5.2: Запуск программы для самостоятельной работы

6 Выводы

При выполнении данной лабораторной работы я приобрела навыки написания программ с использованием циклов и научилась обрабатывать аргументы командной строки.

Список литературы

1. https://esystem.rudn.ru/pluginfile.php/2089095/mod_resource/content/0/%D0%9B%D0%B0%D0%B1%D0%BE%D1%80%D0%B0%D1%82%D0%BE%D1%80%D0%BD%D0%B0%D1%8F%20%D1%80%D0%B0%D0%B1%D0%BE%D1%82%D0%B0%20%E2%84%968.%20%D0%9F%D1%80%D0%BE%D0%B3%D1%80%D0%B0%D0%BC%D0%BC%D0%B8%D1%80%D0%BE%D0%B2%D0%B0%D0%BD%D0%B8%D0%B5%20%D1%86%D0%B8%D0%BA%D0%BB%D0%B0.%20%D0%9E%D0%B1%D1%80%D0%B0%D0%B1%D0%BE%D1%82%D0%BA%D0%B0%20%D0%B0%D1%80%D0%B3%D1%83%D0%BC%D0%B5%D0%BD%D1%82%D0%BE%D0%B2%20%D0%BA%D0%BE%D0%BC%D0%B0%D0%BD%D0%B4%D0%BD%D0%BE%D0%B9%20%D1%81%D1%82%D1%80%D0%BE%D0%BA%D0%B8..pdf
2. <https://esystem.rudn.ru/course/view.php?id=112>
3. <https://esystem.rudn.ru/mod/page/view.php?id=1030492>
4. <https://esystem.rudn.ru/mod/resource/view.php?id=1030496>